

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

“Single Linked List”

disusun Oleh:

Muhammad Fedora Argadyaksa

2511533016

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum *Struktur Data* ini dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada mata kuliah Struktur Data. Dalam laporan ini, penulis memaparkan hasil praktikum yang telah dilakukan, mulai dari konsep dasar hingga implementasi struktur data seperti array, stack, queue, dan linked list dalam pemrograman.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat banyak kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan di masa yang akan datang.

Tidak lupa, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu dalam pelaksanaan praktikum serta penyusunan laporan ini.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya dalam memahami konsep dan penerapan struktur data.

Padang, 5 Mei 2025

M. Fedora Argadyaksa

DAFTAR ISI

KATA PENGANTAR.....	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	4
1.1 Latar Belakang.....	4
1.2 Tujuan.....	4
1.3 Manfaat.....	4
BAB II PEMBAHASAN.....	6
2.1 Pengertian Single Linked List	6
2.2 Implementasi Stack pada Java.....	6
2.2.1 Class Node SLL (NodeSLL_2511533016)	7
2.2.2 Class Pencarian SLL (PencarianSLL_2511533016)	8
2.2.3 Class Tambah SLL (TambahSLL_2511533016)	9
2.2.4 Class Reverse Date (ReverseDate_2511533016).....	11
BAB III PENUTUP	14
3.1 Kesimpulan.....	14
3.2 Saran	14
DAFTAR PUSTAKA.....	16
LAPORAN TUGAS PRAKTIKUM	17
1. Class Pasien (Pasien_2511533016)	17
2. Class Rumah Sakit (RumahSakit_2511533016)	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan data menjadi hal yang sangat penting, terutama ketika berhubungan dengan struktur data yang dinamis. Salah satu struktur data yang sering digunakan adalah **Single Linked List**. Struktur ini merupakan kumpulan elemen (node) yang saling terhubung melalui pointer atau referensi, di mana setiap node hanya menunjuk ke satu node berikutnya.

Single Linked List menjadi alternatif dari array karena memiliki keunggulan dalam pengelolaan memori yang lebih fleksibel. Jika pada array ukuran harus ditentukan di awal, maka pada linked list ukuran dapat bertambah atau berkurang secara dinamis sesuai kebutuhan. Hal ini sangat berguna dalam aplikasi yang membutuhkan manipulasi data secara sering, seperti penambahan dan penghapusan elemen.

Selain itu, pemahaman tentang Single Linked List menjadi dasar penting untuk mempelajari struktur data lain yang lebih kompleks, seperti double linked list, stack, queue, hingga graph. Oleh karena itu, materi ini penting untuk dipelajari oleh mahasiswa atau programmer pemula.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum pekan 5 tentang Single Linked List adalah sebagai berikut:

1. Memahami konsep dasar struktur data Single Linked List
2. Mengetahui cara kerja node dan pointer dalam menghubungkan data.
3. Mampu mengimplementasikan Single Linked List dalam pemrograman.

1.3 Manfaat

Manfaat yang diharapkan dari pelaksanaan praktikum pekan 5 tentang Single Linked List adalah sebagai berikut:

1. Efisiensi Memori.
2. Kemudahan Manipulasi Data.
3. Dasar Pemahaman Struktur Data Lanjutan
4. Meningkatkan Logika Pemrograman

BAB II PEMBAHASAN

2.1 Pengertian Single Linked List

Single Linked List adalah struktur data linear yang terdiri dari sekumpulan elemen yang disebut node, di mana setiap node saling terhubung secara berurutan melalui sebuah pointer. Setiap node memiliki dua bagian utama, yaitu data yang berfungsi untuk menyimpan nilai atau informasi, serta pointer (next) yang berfungsi untuk menyimpan alamat atau referensi ke node berikutnya. Disebut *single* karena setiap node hanya memiliki satu pointer yang mengarah ke node selanjutnya, sehingga hubungan antar node bersifat satu arah (forward).

Struktur ini diawali dari node pertama yang disebut **head**, dan untuk mengakses data harus dilakukan secara berurutan dengan mengikuti pointer dari satu node ke node berikutnya hingga mencapai node terakhir yang menunjuk ke **NULL**, yang menandakan akhir dari rangkaian data. Berbeda dengan array, Single Linked List tidak disimpan dalam memori yang berurutan, sehingga bersifat dinamis dan memungkinkan penambahan maupun penghapusan data dengan lebih fleksibel tanpa harus menentukan ukuran di awal.

Dengan karakteristik tersebut, Single Linked List sangat cocok digunakan dalam kondisi yang membutuhkan manipulasi data secara dinamis, meskipun memiliki keterbatasan dalam hal akses langsung terhadap elemen tertentu karena harus dilakukan secara traversal dari node pertama.

2.2 Implementasi Stack pada Java

Pada praktikum pekan 5 mahasiswa disuruh untuk mengikuti intruksi dari dosen. Yaitu menuliskan kembali kode program yang sudah ada di layar TV. Kode program yang dibuat tentang Single Linked List, di bawah ini kode program yang dituliskan pada praktikum pekan 4.

2.2.1 Class Node SLL (NodeSLL_2511533016)

```

1 package Pekan5_2511533016;
2
3 public class NodeSLL_2511533016 {
4     // node bagian data
5     int data_3016;
6     // Pointer ke node berikutnya
7     NodeSLL_2511533016 next_3016;
8     // Kontruktor menginisialisasi node dengan data
9     public NodeSLL_2511533016(int data_3016) {
10
11         this.data_3016 = data_3016;
12         this.next_3016 = null;
13     }
14 }
15

```

Gambar 2.2.1

Kode program tersebut merupakan implementasi **class node** untuk struktur data *Single Linked List* dalam bahasa Java. Pada baris awal, package `Pekan5_2511533016`; menunjukkan bahwa class ini berada dalam package tertentu. Kemudian didefinisikan class `NodeSLL_2511533016` yang berfungsi sebagai representasi satu node dalam linked list. Di dalam class ini terdapat dua atribut utama, yaitu `int data_3016` yang digunakan untuk menyimpan nilai atau data, serta `NodeSLL_2511533016 next_3016` yang berfungsi sebagai pointer atau referensi ke node berikutnya. Selanjutnya terdapat konstruktor `NodeSLL_2511533016(int data_3016)` yang digunakan untuk menginisialisasi objek node saat pertama kali dibuat, di mana nilai parameter `data_3016` akan disimpan ke dalam atribut `data_3016` menggunakan `this.data_3016 = data_3016`, dan pointer `next_3016` diatur bernilai `null` melalui `this.next_3016 = null`, yang menandakan bahwa node tersebut belum terhubung ke node lain. Secara keseluruhan, kode ini adalah dasar pembentukan elemen dalam *Single Linked List*, di mana setiap node menyimpan data dan alamat node berikutnya.

2.2.2 Class Pencarian SLL (PencarianSLL_2511533016)

```

1 package Pekan5_2511533016;
2
3 public class PencarianSLL_2511533016 {
4     static boolean searchKey_2511533016(NodeSLL_2511533016 head_3016, int key_3016) {
5         NodeSLL_2511533016 curr_3016 = head_3016;
6         while (curr_3016 != null) {
7             if (curr_3016.data_3016 == key_3016)
8                 return true;
9             curr_3016 = curr_3016.next_3016;
10        }
11        return false;
12    }
13    public static void traversal_2511533016(NodeSLL_2511533016 head_3016) {
14        // mulai dari head
15        NodeSLL_2511533016 curr_3016 = head_3016;
16        // telusuri sampai pointer null
17        while (curr_3016 != null) {
18            System.out.print(" " + curr_3016.data_3016);
19            curr_3016 = curr_3016.next_3016;
20        }
21        System.out.println();
22    }
23    public static void main(String[] args) {
24        NodeSLL_2511533016 head_3016 = new NodeSLL_2511533016(14);
25        head_3016.next_3016 = new NodeSLL_2511533016(21);
26        head_3016.next_3016.next_3016 = new NodeSLL_2511533016(13);
27        head_3016.next_3016.next_3016.next_3016 = new NodeSLL_2511533016(30);
28        head_3016.next_3016.next_3016.next_3016.next_3016 = new NodeSLL_2511533016(10);
29        System.out.print("Penelusuran SLL : ");
30        traversal_2511533016(head_3016);
31        // data yang akan dicari
32        int key_3016 = 30;
33        System.out.println("cari data" + key_3016 + " = ");
34        if (searchKey_2511533016(head_3016, key_3016))
35            System.out.println("Ketemu");
36        else
37            System.out.println("tidak ada");
38    }
39 }

```

Penelusuran SLL : 14 21 13 30 10
 cari data30 =
 Ketemu

Gambar 2.2.2

Kode program tersebut merupakan implementasi operasi **pencarian (search)** dan **penelusuran (traversal)** pada struktur data *Single Linked List* menggunakan bahasa Java. Program diawali dengan deklarasi package `Pekan5_2511533016` dan class `PencarianSLL_2511533016`. Di dalam class ini terdapat method `searchKey_2511533016` yang bertipe boolean dan berfungsi untuk mencari suatu nilai (key) dalam linked list. Method ini menerima parameter berupa `head_3016` (node awal) dan `key_3016` (data yang dicari), kemudian menggunakan variabel bantu `curr_3016` untuk menelusuri node satu per satu dengan perulangan `while (curr_3016 != null)`. Jika data pada node yang sedang diperiksa sama dengan key (`curr_3016.data_3016 == key_3016`), maka method akan mengembalikan nilai `true`, sedangkan jika seluruh node sudah diperiksa dan tidak ditemukan, maka akan mengembalikan `false`.

Selain itu, terdapat method `traversal_2511533016` yang bertujuan untuk menampilkan seluruh isi linked list. Method ini juga menggunakan variabel `curr_3016` yang dimulai dari head, lalu melakukan perulangan hingga mencapai node terakhir (null), sambil mencetak data setiap node menggunakan `System.out.print`. Setelah semua data ditampilkan, program mencetak baris baru.

Pada method main, program membuat sebuah linked list secara manual dengan membuat node pertama bernilai 14 sebagai head, kemudian menghubungkannya dengan node-node berikutnya yang berisi nilai 21, 13, 30, dan 10 melalui pointer next_3016. Setelah itu, program menampilkan isi linked list dengan memanggil method traversal. Selanjutnya ditentukan nilai yang akan dicari yaitu 30, lalu method searchKey_2511533016 dipanggil untuk mengecek apakah nilai tersebut ada di dalam linked list. Jika ditemukan, program akan menampilkan output "Ketemu", sedangkan jika tidak ditemukan akan menampilkan "tidak ada". Secara keseluruhan, kode ini menunjukkan cara membuat linked list sederhana, menampilkan isinya, serta melakukan pencarian data di dalamnya.

2.2.3 Class Tambah SLL (TambahSLL_2511533016)

```

1 package Pekanbaru_2511533016;
2
3 public class TambahSLL_2511533016 {
4     public static NodeSLL_2511533016 insertAtFront_2511533016(NodeSLL_2511533016 head_3016, int value_3016) {
5         NodeSLL_2511533016 new_node_3016 = new NodeSLL_2511533016(value_3016);
6         new_node_3016.next_3016 = head_3016;
7         return new_node_3016;
8     }
9
10    // fungsi menambahkan node di akhir SLL
11    public static NodeSLL_2511533016 insertAtEnd_2511533016(NodeSLL_2511533016 head_3016, int value_3016) {
12        // buat sebuah node dengan sebuah nilai
13        NodeSLL_2511533016 new_node_3016 = new NodeSLL_2511533016(value_3016);
14        // jika list kosong maka node jadi head
15        if (head_3016 == null) {
16            return new_node_3016;
17        }
18        // simpun head ke variabel sementara
19        NodeSLL_2511533016 last_3016 = head_3016;
20        // lanjut ke node akhir
21        while (last_3016.next_3016 != null) {
22            last_3016 = last_3016.next_3016;
23        }
24        // ubah pointer
25        last_3016.next_3016 = new_node_3016;
26        return head_3016;
27    }
28
29    static NodeSLL_2511533016 GetNode_2511533016(int data_3016) {
30        return new NodeSLL_2511533016(data_3016);
31    }
32
33    static NodeSLL_2511533016 insertPos_2511533016(NodeSLL_2511533016 headNode_3016, int position_3016, int value_3016) {
34        if (position_3016 < 1) {
35            System.out.println("Invalid position");
36            return headNode_3016;
37        }
38        if (position_3016 == 1) {
39            NodeSLL_2511533016 new_node_3016 = new NodeSLL_2511533016(value_3016);
40            new_node_3016.next_3016 = headNode_3016;
41            return new_node_3016;
42        }
43        else {
44            (position_3016 - 1) {
45                while (position_3016 == 1) {
46                    NodeSLL_2511533016 new_node_3016 = GetNode_2511533016(value_3016);
47                    new_node_3016.next_3016 = headNode_3016.next_3016;
48                    headNode_3016.next_3016 = new_node_3016;
49                    break;
50                }
51                headNode_3016 = headNode_3016.next_3016;
52            }
53            if (position_3016 != 1) {
54                System.out.println("Posisi di luar jangkauan");
55            }
56            return headNode_3016;
57        }
58    }
59
60    public static void printList_2511533016(NodeSLL_2511533016 head_3016) {
61        NodeSLL_2511533016 curr_3016 = head_3016;
62        while (curr_3016.next_3016 != null) {
63            System.out.print(curr_3016.data_3016 + " ");
64            curr_3016 = curr_3016.next_3016;
65        }
66        if (curr_3016.next_3016 == null) {
67            System.out.print(curr_3016.data_3016);
68            System.out.println();
69        }
70    }
71
72    public static void main(String[] args) {
73        // buat linked list 2-3-5-8-0
74        NodeSLL_2511533016 head_3016 = new NodeSLL_2511533016(2);
75        head_3016.next_3016 = new NodeSLL_2511533016(3);
76        head_3016.next_3016.next_3016 = new NodeSLL_2511533016(5);
77        head_3016.next_3016.next_3016.next_3016 = new NodeSLL_2511533016(8);
78        // link list jadi
79        System.out.print("Senarai berantai awal : ");
80        printList_2511533016(head_3016);
81        // tambahkan node baru di depan
82        System.out.print("Tambah 1 simpul di depan : ");
83        int data_3016 = 1;
84        head_3016 = insertAtFront_2511533016(head_3016, data_3016);
85        // link update list
86        printList_2511533016(head_3016);
87        // tambahkan node baru di belakang
88        System.out.print("Tambah 1 simpul di belakang : ");
89        int data2_3016 = 7;
90        head_3016 = insertAtEnd_2511533016(head_3016, data2_3016);
91        // link update list
92        printList_2511533016(head_3016);
93        // tambahkan 1 simpul ke data 4 :
94        int data3_3016 = 4;
95        int pos_3016 = 4;
96    }
97 }

```

```

89     head_3016 = insertPos(head_3016, pos_3016, data3_3016);
90     // cetak update list
91     printList_2511533016(head_3016);
92 }
93
94 }
95

```

Senarai berantai awal : 2-->3-->5-->6
 tambah 1 simpul di depan : 1-->2-->3-->5-->6
 Tambah 1 simpul di belakang : 1-->2-->3-->5-->6-->7
 tambah 1 simpul ke data 4 : 1-->2-->3-->4-->5-->6-->7

Gambar 2.2.3

Kode program tersebut merupakan implementasi operasi **penambahan (insert)** pada struktur data *Single Linked List* dalam bahasa Java. Program diawali dengan deklarasi package `Pekan5_2511533016` dan class `TambahSLL_2511533016` yang berisi beberapa method untuk menambahkan node pada berbagai posisi, yaitu di depan, di akhir, dan pada posisi tertentu, serta method untuk mencetak isi linked list.

Method `insertAtFront_2511533016` digunakan untuk menambahkan node di bagian depan (head). Cara kerjanya adalah dengan membuat node baru menggunakan nilai yang diberikan, kemudian pointer next dari node baru diarahkan ke head lama, dan node baru tersebut dikembalikan sebagai head yang baru. Selanjutnya, method `insertAtEnd_2511533016` berfungsi untuk menambahkan node di bagian akhir linked list. Jika linked list masih kosong (`head == null`), maka node baru langsung dijadikan head. Jika tidak kosong, program akan menelusuri hingga node terakhir menggunakan perulangan `while`, kemudian menghubungkan node terakhir tersebut dengan node baru melalui pointer `next`.

Method `GetNode_2511533016` merupakan fungsi sederhana untuk membuat node baru berdasarkan data yang diberikan. Kemudian method `insertPos` digunakan untuk menambahkan node pada posisi tertentu dalam linked list. Jika posisi bernilai 1, maka proses sama seperti insert di depan. Jika posisi lebih dari 1, maka program akan menelusuri node hingga posisi yang diinginkan, lalu menyisipkan node baru dengan cara mengatur pointer agar node baru berada di antara node sebelumnya dan node berikutnya. Jika posisi tidak valid atau di luar jangkauan, program akan menampilkan pesan kesalahan.

Selain itu, terdapat method `printList_2511533016` yang berfungsi untuk menampilkan seluruh isi linked list. Method ini menelusuri setiap node dari head

hingga node terakhir, lalu mencetak data dengan format berantai (menggunakan tanda panah “->”), dan diakhiri dengan baris baru.

Pada method main, program membuat linked list awal secara manual dengan isi data $2 \rightarrow 3 \rightarrow 5 \rightarrow 6$. Kemudian program menampilkan list awal, lalu melakukan beberapa operasi penambahan, yaitu menambahkan node dengan nilai 1 di depan, menambahkan node dengan nilai 7 di akhir, dan menambahkan node dengan nilai 4 pada posisi ke-4. Setelah setiap operasi, linked list dicetak kembali untuk menunjukkan hasil perubahan. Secara keseluruhan, kode ini menggambarkan bagaimana melakukan manipulasi data pada Single Linked List, khususnya operasi penambahan node di berbagai posisi serta menampilkan hasilnya.

2.2.4 Class Reverse Date (ReverseDate_2511533016)

```

1 package Peko5_2511533016;
2
3 public class ReverseDate_2511533016 {
4     // Fungsi untuk menghapus head
5     public static NodeSLL_2511533016 deleteHead_2511533016(NodeSLL_2511533016 head_3016) {
6         // Jika SLL kosong
7         if (head_3016 == null)
8             return null;
9         // Menghapus head ke node berikutnya
10        head_3016 = head_3016.next_3016;
11        // Return head baru
12        return head_3016;
13    }
14    // Fungsi untuk menghapus node terakhir SLL
15    public static NodeSLL_2511533016 removeLastNode_2511533016(NodeSLL_2511533016 head_3016) {
16        // Jika list kosong, return null
17        if (head_3016 == null) {
18            return null;
19        }
20        // Jika list satu node, hapus node dan return null
21        if (head_3016.next_3016 == null) {
22            return null;
23        }
24        // Temporal node terakhir ke dua
25        NodeSLL_2511533016 secondLast = head_3016;
26        while (secondLast.next_3016 != null) {
27            secondLast = secondLast.next_3016;
28        }
29        // Hapus node terakhir
30        secondLast.next_3016 = null;
31        return head_3016;
32    }
33    // Fungsi untuk menambahkan node di posisi tertentu
34    public static NodeSLL_2511533016 insertNode_2511533016(NodeSLL_2511533016 head_3016, int position_3016) {
35        NodeSLL_2511533016 temp_3016 = head_3016;
36        NodeSLL_2511533016 prev_3016 = null;
37        // Jika linked list null
38        if (temp_3016 == null)
39            return head_3016;
40        // Kasus 1 : head dihapus
41        if (position_3016 == 1) {
42            head_3016 = temp_3016.next_3016;
43            return head_3016;
44        }
45        // Kasus 2 : menambahkan node di tengah
46        // Temporal ke node baru di head
47        for (int i_3016 = 1; temp_3016 != null && i_3016 < position_3016; i_3016++) {
48            prev_3016 = temp_3016;
49            temp_3016 = temp_3016.next_3016;
50        }
51        // Jika dilewatkan, hapus node
52        if (temp_3016 != null) {
53            prev_3016.next_3016 = temp_3016.next_3016;
54        } else {
55            System.out.println("data tidak ada");
56            return head_3016;
57        }
58        // Fungsi untuk menambahkan node
59        public static NodeSLL_2511533016 printList_2511533016(NodeSLL_2511533016 head_3016) {
60            NodeSLL_2511533016 curr_3016 = head_3016;
61            while (curr_3016.next_3016 != null) {
62                System.out.print(curr_3016.data_3016 + " -> ");
63                curr_3016 = curr_3016.next_3016;
64            }
65            System.out.print(curr_3016.data_3016 + "\n");
66            System.out.println();
67        }
68        // Selama main
69        public static void main(String[] args) {
70            // Buat SLL 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
71            NodeSLL_2511533016 head_3016 = new NodeSLL_2511533016(1);
72            head_3016.next_3016 = new NodeSLL_2511533016(2);
73            head_3016.next_3016.next_3016 = new NodeSLL_2511533016(3);
74            head_3016.next_3016.next_3016.next_3016 = new NodeSLL_2511533016(4);
75            head_3016.next_3016.next_3016.next_3016.next_3016 = new NodeSLL_2511533016(5);
76            head_3016.next_3016.next_3016.next_3016.next_3016.next_3016 = new NodeSLL_2511533016(6);
77            // cetak list awal
78            System.out.println("list awal : ");
79            printList_2511533016(head_3016);
80            // hapus head
81            head_3016 = deleteHead_2511533016(head_3016);
82            System.out.println("list setelah head dihapus : ");
83            printList_2511533016(head_3016);
84            // hapus node terakhir
85            head_3016 = removeLastNode_2511533016(head_3016);
86            System.out.println("list setelah simpul terakhir dihapus : ");
87            printList_2511533016(head_3016);
88            // Deleting node at position 2
89            int position_3016 = 2;
90            head_3016 = deleteNode_2511533016(head_3016, position_3016);
91            // print list after deletion
92            System.out.println("list setelah posisi 2 dihapus : ");
93            printList_2511533016(head_3016);
94            printList_2511533016(head_3016);
95        }
96    }
97 }
98
99 }
100
101 }
102
103 }
104

```

```

List awal :
1-->2-->3-->4-->5-->6
List setelah head dihapus :
2-->3-->4-->5-->6
List setelah simpul terakhir dihapus :
2-->3-->4-->5-->6
List setelah posisi 2 dihapus :
2-->4-->5-->6

```

Gambar 2.2.4

Kode program tersebut merupakan implementasi operasi **penghapusan (delete)** pada struktur data *Single Linked List* menggunakan bahasa Java. Program berada dalam package `Pekan5_2511533016` dan memiliki class `HapusSLL_2511533016` yang berisi beberapa method untuk menghapus node dari berbagai posisi, yaitu di awal (head), di akhir, dan pada posisi tertentu, serta method untuk menampilkan isi linked list.

Method `deleteHead_2511533016` berfungsi untuk menghapus node pertama (head). Jika linked list kosong (`head == null`), maka method langsung mengembalikan null. Jika tidak, head akan dipindahkan ke node berikutnya (`head.next`), sehingga node pertama sebelumnya otomatis terhapus, lalu mengembalikan head yang baru. Selanjutnya, method `removeLastNode_2511533016` digunakan untuk menghapus node terakhir. Jika linked list kosong atau hanya memiliki satu node, maka hasilnya adalah null. Jika lebih dari satu node, program menggunakan variabel `secondLast` untuk menelusuri hingga node sebelum terakhir, kemudian memutus hubungan node terakhir dengan mengatur `secondLast.next = null`.

Method `deleteNode_2511533016` digunakan untuk menghapus node pada posisi tertentu. Program menggunakan dua variabel bantu, yaitu `temp` sebagai penunjuk node yang sedang diperiksa dan `prev` sebagai node sebelumnya. Jika posisi yang dihapus adalah posisi pertama, maka sama seperti menghapus head. Jika bukan, program melakukan perulangan untuk mencapai posisi yang diinginkan. Jika node ditemukan, maka pointer dari node sebelumnya (`prev.next`) akan diarahkan ke node setelah node yang dihapus, sehingga node tersebut terlepas dari linked list. Jika posisi tidak ditemukan, program akan menampilkan pesan "Data tidak ada".

Method `printList_2511533016` digunakan untuk mencetak isi linked list. Program menelusuri setiap node dari head hingga node terakhir, mencetak data

dengan format berantai menggunakan tanda panah (-->), lalu menampilkan node terakhir tanpa panah dan diakhiri dengan baris baru.

Pada method main, program membuat linked list secara manual dengan isi data $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow \text{null}$. Kemudian program menampilkan list awal, dilanjutkan dengan operasi penghapusan secara bertahap, yaitu menghapus node pertama (head), menghapus node terakhir, dan menghapus node pada posisi ke-2. Setelah setiap proses penghapusan, linked list ditampilkan kembali untuk menunjukkan perubahan yang terjadi. Secara keseluruhan, kode ini menjelaskan bagaimana melakukan berbagai operasi penghapusan pada Single Linked List serta menampilkan hasilnya setelah setiap operasi dilakukan.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan seluruh kode program yang telah dibahas (pembuatan node, pencarian, penambahan, dan penghapusan), dapat disimpulkan bahwa *Single Linked List* merupakan struktur data dinamis yang tersusun dari node-node yang saling terhubung melalui pointer. Implementasi program menunjukkan bagaimana data dapat dikelola secara fleksibel melalui berbagai operasi utama, yaitu traversal (penelusuran), search (pencarian), insert (penambahan), dan delete (penghapusan). Setiap operasi memiliki mekanisme yang berbeda namun tetap berpusat pada manipulasi pointer antar node. Penambahan dapat dilakukan di depan, belakang, maupun posisi tertentu dengan cara mengatur hubungan node baru ke node lain, sedangkan penghapusan dilakukan dengan memutus hubungan node yang ingin dihapus dari rangkaian linked list. Pencarian dan traversal dilakukan secara berurutan dari head hingga node terakhir karena tidak adanya akses langsung seperti pada array.

Secara keseluruhan, kode-kode tersebut menggambarkan bahwa pengelolaan Single Linked List sangat bergantung pada ketelitian dalam mengatur pointer agar struktur data tetap konsisten. Pemahaman ini menjadi dasar penting dalam mempelajari struktur data lanjutan serta meningkatkan kemampuan logika dalam pemrograman.

3.2 Saran

Adapun saran yang dapat diberikan berdasarkan materi ini adalah sebagai berikut:

1. Pastikan setiap perubahan pada linked list (insert/delete) dilakukan dengan benar agar tidak terjadi kehilangan data atau error (misalnya null pointer).
2. Sebaiknya tambahkan pengecekan posisi atau kondisi list agar program lebih aman, seperti validasi posisi saat insert atau delete.
3. Pisahkan fungsi-fungsi ke dalam class yang berbeda (misalnya class `LinkedList` dan `Node`) agar struktur program lebih terorganisir.
4. Kembangkan program dengan fitur tambahan seperti menghitung jumlah

node, mencari nilai maksimum/minimum, atau reverse linked list.

5. Cobalah berbagai skenario (list kosong, satu node, banyak node) untuk memahami cara kerja linked list secara lebih mendalam.

DAFTAR PUSTAKA

- [1] T. H. Cormen, C. E. Leiserson dan R. L. Rivest, “Introducing to Algorithms,” 2009. [Online].
- [2] M. T. Goodrich, R. Tamassia dan M. H. Goldwasser, “Data Structures and Algorithms in Java,” 2014. [Online].

LAPORAN TUGAS PRAKTIKUM

SOAL:

Mahasiswa diminta untuk mengimplementasikan struktur data Queue (Antrian) menggunakan array dalam bahasa Java. Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip FIFO (First In First Out)

1. Class Pasien (Pasien_2511533016)

```

1 package Pekan5_2511533016;
2
3 public class Pasien_2511533016 {
4     String namaPasien_3016;
5     String penyakit_3016;
6     int nomorAntrian_3016;
7     Pasien_2511533016 next_3016;
8
9     // Constructor
10    public Pasien_2511533016(String nama_3016, String penyakit_3016, int nomor_3016) {
11        this.namaPasien_3016 = nama_3016;
12        this.penakit_3016 = penyakit_3016;
13        this.nomorAntrian_3016 = nomor_3016;
14        this.next_3016 = null;
15    }
16
17    // Getter
18    public String getNama_3016() {
19        return namaPasien_3016;
20    }
21
22    public String getPenyakit_3016() {
23        return penyakit_3016;
24    }
25
26    public int getNomor_3016() {
27        return nomorAntrian_3016;
28    }
29
30    public Pasien_2511533016 getNext_3016() {
31        return next_3016;
32    }
33
34    // Setter
35    public void setNext_3016(Pasien_2511533016 next) {
36        this.next_3016 = next;
37    }
38 }

```

Gambar 1

Kode program di atas merupakan sebuah kelas Java bernama Pasien_2511533016 yang digunakan untuk merepresentasikan data pasien pada struktur data **Single Linked List**. Program ini berada di dalam package Pekan5_2511533016. Kelas ini menyimpan beberapa atribut penting yaitu namaPasien_3016 bertipe String untuk menyimpan nama pasien, penyakit_3016 bertipe String untuk menyimpan jenis penyakit pasien, nomorAntrian_3016 bertipe int untuk menyimpan nomor antrian pasien, dan next_3016 bertipe Pasien_2511533016 yang digunakan sebagai penunjuk ke node berikutnya pada linked list.

Pada bagian constructor Pasien_2511533016(String nama_3016, String penyakit_3016, int nomor_3016), constructor ini berfungsi untuk menginisialisasi objek ketika data pasien baru dibuat. Nilai parameter nama_3016, penyakit_3016, dan nomor_3016 akan dimasukkan ke atribut kelas menggunakan keyword this.

Selain itu, atribut `next_3016` diatur bernilai null karena saat node pertama kali dibuat, node tersebut belum terhubung dengan node lain.

Selanjutnya terdapat beberapa method getter yang digunakan untuk mengambil atau menampilkan nilai dari atribut. Method `getNama_3016()` mengembalikan nilai nama pasien, method `getPenyakit_3016()` mengembalikan data penyakit pasien, method `getNomor_3016()` mengembalikan nomor antrian pasien, dan method `getNext_3016()` mengembalikan alamat node berikutnya yang disimpan pada variabel `next_3016`. Getter ini berguna agar data dapat diakses dari luar kelas tanpa harus langsung memanggil atributnya.

Pada bagian terakhir terdapat method setter yaitu `setNext_3016(Pasien_2511533016 next)`. Method ini digunakan untuk menghubungkan node saat ini dengan node berikutnya pada linked list. Nilai parameter `next` akan disimpan ke variabel `next_3016` sehingga node dapat saling terhubung membentuk rantai linked list.

Secara keseluruhan, kode program ini digunakan sebagai kelas node dalam implementasi Single Linked List data pasien. Setiap objek pasien menyimpan informasi nama, penyakit, nomor antrian, serta alamat node berikutnya. Constructor digunakan untuk membuat data pasien baru, getter digunakan untuk mengambil data pasien, sedangkan setter digunakan untuk menghubungkan antar node sehingga data pasien dapat disusun secara berantai dalam linked list.

2. Class Rumah Sakit (RumahSakit_2511533016)

```

1 package RumahSakit_2511533016;
2 import java.util.Scanner;
3
4 public class RumahSakit_2511533016 {
5     Pasien_2511533016 head_3016;
6     int counter_3016 = 0;
7
8     // Terverifikasi (Daftar Pasien)
9     public void daftarPasien_2511533016(String nama_3016, String penyakit_3016) {
10         counter_3016++;
11         Pasien_2511533016 baru_3016 = new Pasien_2511533016(nama_3016, penyakit_3016, counter_3016);
12
13         if (head_3016 == null) {
14             head_3016 = baru_3016;
15         } else {
16             Pasien_2511533016 temp_3016 = head_3016;
17             while (temp_3016.getNext_3016() != null) {
18                 temp_3016 = temp_3016.getNext_3016();
19             }
20             temp_3016.setNext_3016(baru_3016);
21         }
22         System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " + counter_3016);
23     }
24
25     // Delete Head (Hapus Pasien)
26     public void penggilPasien_2511533016() {
27         if (head_3016 == null) {
28             System.out.println("Antrian kosong!");
29         } else {
30             System.out.println("Panggil Pasien!");
31             System.out.println("Nama: " + head_3016.getNama_3016());
32             System.out.println("Keluhan: " + head_3016.getPenyakit_3016());
33             head_3016 = head_3016.getNext_3016();
34         }
35     }
36
37     // Display
38     public void tampilkanAntrian_2511533016() {
39         if (head_3016 == null) {
40             System.out.println("Antrian kosong!");
41         } else {
42             Pasien_2511533016 temp_3016 = head_3016;
43             while (temp_3016 != null) {
44                 System.out.println("No: " + temp_3016.getNomor_3016() +
45                     " | Nama: " + temp_3016.getNama_3016() +
46                     " | Keluhan: " + temp_3016.getPenyakit_3016());
47                 temp_3016 = temp_3016.getNext_3016();
48             }
49         }
50     }
51
52     // Search
53     public void cariPasien_2511533016(String nama) {
54         Pasien_2511533016 temp_3016 = head_3016;
55         boolean ditemukan_3016 = false;
56
57         while (temp_3016 != null) {
58             if (temp_3016.getNama_3016().equalsIgnoreCase(nama)) {
59                 System.out.println("Pasien ditemukan!");
60                 System.out.println("No: " + temp_3016.getNomor_3016());
61                 System.out.println("Nama: " + temp_3016.getNama_3016());
62                 System.out.println("Keluhan: " + temp_3016.getPenyakit_3016());
63                 ditemukan_3016 = true;
64                 break;
65             }
66             temp_3016 = temp_3016.getNext_3016();
67         }
68
69         if (ditemukan_3016) {
70             System.out.println("Pasien tidak ditemukan.");
71         }
72     }
73
74     // Status
75     public void cekStatus_2511533016() {
76         if (head_3016 == null) {
77             System.out.println("Antrian kosong!");
78         } else {
79             int jumlah_3016 = 0;
80             Pasien_2511533016 temp_3016 = head_3016;
81
82             while (temp_3016 != null) {
83                 jumlah_3016++;
84                 temp_3016 = temp_3016.getNext_3016();
85             }
86             System.out.println("Jumlah Pasien: " + jumlah_3016);
87         }
88     }
89
90     // Pasien Terdepan
91     public void lihatTerdepan_2511533016() {
92         if (head_3016 == null) {
93             System.out.println("Pasien terdepan: " + head_3016.getNama_3016());
94         }
95     }
96
97     // Main Menu
98     public static void main(String[] args) {
99         Scanner input_3016 = new Scanner(System.in);
100         RumahSakit_2511533016 rs_3016 = new RumahSakit_2511533016();
101         int pilihan_3016;
102
103         do {
104             System.out.println("===== Antrian Rumah Sakit NUR 202233016 =====");
105             System.out.println("1. Daftarkan Pasien");
106             System.out.println("2. Panggil Pasien");
107             System.out.println("3. Tampilkan Antrian");
108             System.out.println("4. Cari Pasien");
109             System.out.println("5. Cek Status");
110             System.out.println("6. Keluar");
111             System.out.print("Pilih menu: ");
112             pilihan_3016 = input_3016.nextInt();
113             input_3016.nextLine();
114
115             switch (pilihan_3016) {
116                 case 1:
117                     System.out.print("Nama: ");
118                     String nama = input_3016.nextLine();
119                     System.out.print("Keluhan: ");
120                     String keluhan = input_3016.nextLine();
121                     rs_3016.daftarPasien_2511533016(nama, keluhan);
122                     break;
123                 case 2:
124                     rs_3016.panggilPasien_2511533016();
125                     break;
126                 case 3:
127                     rs_3016.tampilkanAntrian_2511533016();
128                     break;
129                 case 4:
130                     System.out.print("Masukkan nama: ");
131                     String cari = input_3016.nextLine();
132                     rs_3016.cariPasien_2511533016(cari);
133                     break;
134                 case 5:
135                     rs_3016.cekStatus_2511533016();
136                     break;
137                 case 6:
138                     System.out.println("Keluar");
139                     break;
140                 default:
141                     System.out.println("Pilihan tidak valid!");
142             }
143         } while (pilihan_3016 != 6);
144
145         input_3016.close();
146     }
147 }

```

Gambar 2

Kode program di atas merupakan program Java berbasis **Single Linked List** yang digunakan untuk mengelola antrian pasien rumah sakit. Program ini

berada pada package `Pekan5_2511533016` dan menggunakan class `RumahSakit_2511533016` sebagai pengelola utama data pasien. Pada bagian awal program terdapat `import java.util.Scanner;` yang digunakan untuk menerima input dari pengguna melalui keyboard.

Di dalam class `RumahSakit_2511533016` terdapat variabel `head_3016` bertipe `Pasien_2511533016` yang berfungsi sebagai penunjuk node pertama pada linked list atau antrian pasien. Selain itu terdapat variabel `counter_3016` bertipe `int` yang digunakan untuk menghitung nomor antrian pasien secara otomatis setiap kali pasien baru didaftarkan.

Bagian method `daftarPasien_2511533016(String nama_3016, String penyakit_3016)` digunakan untuk menambahkan pasien baru ke dalam antrian. Pertama, nilai `counter_3016` ditambah satu untuk membuat nomor antrian baru. Setelah itu dibuat objek baru bernama `baru_3016` menggunakan constructor dari class `Pasien_2511533016`. Program kemudian memeriksa apakah `head_3016` bernilai null. Jika kosong, maka node baru langsung dijadikan head. Namun jika antrian sudah memiliki isi, program menggunakan variabel sementara `temp_3016` untuk menelusuri linked list menggunakan perulangan `while` hingga mencapai node terakhir. Setelah mencapai node terakhir, node baru dihubungkan menggunakan method `setNext_3016()`. Di akhir method, program menampilkan pesan bahwa pasien berhasil didaftarkan beserta nomor antriannya.

Method `panggilPasien_2511533016()` digunakan untuk menghapus pasien paling depan dalam antrian atau proses delete head pada linked list. Program terlebih dahulu memeriksa apakah `head_3016` kosong. Jika kosong maka muncul pesan “Antrian kosong!”. Jika tidak kosong, program menampilkan data pasien terdepan seperti nama dan keluhan menggunakan getter. Setelah itu, `head_3016` dipindahkan ke node berikutnya menggunakan `head_3016 = head_3016.getNext_3016()`; sehingga pasien paling depan otomatis terhapus dari antrian.

Method `tampilkanAntrian_2511533016()` digunakan untuk menampilkan seluruh data pasien dalam antrian. Program memeriksa apakah antrian kosong. Jika tidak kosong, program menggunakan variabel `temp_3016` untuk menelusuri setiap node dari head hingga akhir linked list menggunakan perulangan `while`. Pada setiap node, program menampilkan nomor antrian, nama pasien, dan keluhan pasien.

Method `cariPasien_2511533016(String nama)` digunakan untuk mencari pasien berdasarkan nama. Program menggunakan variabel `temp_3016` untuk menelusuri linked list dan variabel boolean `ditemukan_3016` untuk menandai apakah data ditemukan atau tidak. Di dalam perulangan `while`, program membandingkan nama pasien menggunakan method `equalsIgnoreCase()` agar pencarian tidak sensitif terhadap huruf besar dan kecil. Jika data ditemukan, program menampilkan informasi pasien berupa nomor antrian, nama, dan keluhan, kemudian `break` digunakan untuk menghentikan pencarian. Jika setelah penelusuran data tidak ditemukan, maka program menampilkan pesan “Pasien tidak ditemukan”.

Method `cekStatus_2511533016()` digunakan untuk melihat status antrian pasien. Program memeriksa apakah antrian kosong. Jika tidak kosong, program menghitung jumlah pasien menggunakan perulangan `while` dengan variabel `jumlah_3016`. Variabel `temp_3016` digunakan untuk menelusuri seluruh node

hingga akhir linked list sambil menambahkan jumlah pasien. Setelah selesai, program menampilkan total pasien dan nama pasien yang berada paling depan pada antrian menggunakan `head_3016.getNama_3016()`.

Pada bagian main, program membuat objek Scanner bernama `input_3016` untuk membaca input pengguna dan objek `RumahSakit_2511533016` bernama `rs_3016` untuk menjalankan semua method yang ada. Variabel `pilihan_3016` digunakan untuk menyimpan pilihan menu pengguna. Program menggunakan perulangan `do-while` agar menu terus tampil sampai pengguna memilih keluar.

Di dalam menu terdapat beberapa pilihan yaitu:

1. Mendaftarkan pasien baru,
2. Memanggil pasien,
3. Menampilkan seluruh antrian,
4. Mencari pasien,
5. Mengecek status antrian,
6. Keluar dari program.

Program menggunakan `switch-case` untuk menjalankan menu sesuai pilihan pengguna. Pada case 1, pengguna diminta memasukkan nama dan keluhan pasien lalu data dimasukkan ke linked list menggunakan method `daftarPasien_2511533016()`. Pada case 2 program memanggil pasien terdepan menggunakan method `panggilPasien_2511533016()`. Pada case 3 program menampilkan seluruh antrian menggunakan `tampilkanAntrian_2511533016()`. Pada case 4 pengguna memasukkan nama pasien yang ingin dicari lalu program menjalankan `cariPasien_2511533016()`. Pada case 5 program menjalankan `cekStatus_2511533016()` untuk melihat jumlah pasien dan pasien terdepan. Pada case 6 program menampilkan pesan keluar dan menghentikan perulangan. Jika pengguna memasukkan pilihan selain menu yang tersedia, maka program menampilkan pesan "Pilihan tidak valid!".

Secara keseluruhan, program ini merupakan implementasi struktur data Single Linked List untuk sistem antrian rumah sakit. Program mampu melakukan operasi insert di akhir linked list untuk pendaftaran pasien, delete head untuk memanggil pasien, traversal untuk menampilkan dan mencari data pasien, serta menghitung jumlah data dalam antrian. Dengan penggunaan linked list, data pasien dapat dikelola secara dinamis dan terstruktur sesuai urutan antrian.

OUTPUT :

MENAMBAHKAN PASIEN DAN MELIHAT ANTRIAN PASIEN :

```

=== Antrian Rumah Sakit NIM: 2511533016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 1
Nama: Fedo
Keluhan: Asam Lambung
Pasien berhasil didaftarkan! Nomor Antrian: 1

=== Antrian Rumah Sakit NIM: 2511533016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 1
Nama: Dennis
Keluhan: Demam
Pasien berhasil didaftarkan! Nomor Antrian: 2

=== Antrian Rumah Sakit NIM: 2511533016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 3
No: 1 | Nama: Fedo | Keluhan: Asam Lambung
No: 2 | Nama: Dennis | Keluhan: Demam

```

CARI PASIEN :

```

=== Antrian Rumah Sakit NIM: 2511533016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 4
Masukkan nama: Fedo
Pasien ditemukan!
No: 1
Nama: Fedo
Keluhan: Asam Lambung

```

CEK STATUS :

```

=== Antrian Rumah Sakit NIM: 2511533016 ===
1. Daftarkan Pasien
2. Panggil Pasien
3. Tampilkan Antrian
4. Cari Pasien
5. Cek Status
6. Keluar
Pilihan: 5
Jumlah Pasien: 2
Pasien terdepan: Fedo

```

PANGGIL PASIEN DAN MELIHAT ANTRIAN :

```
=== Antrian Rumah Sakit NIM: 2511533016 ===  
1. Daftarkan Pasien  
2. Panggil Pasien  
3. Tampilkan Antrian  
4. Cari Pasien  
5. Cek Status  
6. Keluar  
Pilihan: 2  
Memanggil Pasien:  
Nama: Fedo  
Keluhan: Asam Lambung  
  
=== Antrian Rumah Sakit NIM: 2511533016 ===  
1. Daftarkan Pasien  
2. Panggil Pasien  
3. Tampilkan Antrian  
4. Cari Pasien  
5. Cek Status  
6. Keluar  
Pilihan: 3  
No: 2 | Nama: Dennis | Keluhan: Demam
```